

KNMP VisitHub — Project Documentation

Project Name: KNMP VisitHub **Subtitle:** A Web-Based Booking and Visit Planning System for the Kwame Nkrumah Memorial Park **Author:** Manus AI **Date:** August 2026 **Version:** 1.0 (77 passing tests)

1. Introduction

KNMP VisitHub is a web-based booking and visit-planning application for the Kwame Nkrumah Memorial Park (KNMP) in Accra, Ghana. It gives visitors a single place to explore the park's attractions, understand applicable fees, reserve visit dates with unique booking references, and organise their activities into a personal itinerary. On the administrative side, it equips park staff with tools to manage attractions, visitor fee categories, bookings, check-ins, users, and operational analytics.

The system was conceived to solve a practical problem: visitors planning a trip to the park needed scattered information about attractions, fees, opening hours, and planned activities, with no central platform to make a booking or receive confirmation. KNMP VisitHub closes that gap by combining an attractions explorer, an end-to-end booking flow with automatic cost calculation, printable PDF tickets with QR codes, and a staff dashboard covering the full operational lifecycle of a visit.

2. Problem Statement and Proposed Solution

The problem statement identified two pain points. First, prospective visitors had no reliable, centralised source of information about KNMP attractions, pricing, or the booking process, forcing them to rely on word of mouth or physical visits to obtain details. Second, the park had no digital mechanism for capturing reservations, projecting daily attendance, or managing arrivals at the gate, which limited its ability to plan staffing and avoid overcrowding.

The proposed solution is a role-based web application with two personas. The **Visitor** persona can browse and search attractions, view prices, select visit dates (including multi-day stays), choose visitor categories and quantities, receive a live cost summary, confirm a booking, and receive a unique reference and PDF ticket by email. The **Administrator** persona can manage attractions, visitor categories, booking statuses, user accounts, park-wide settings, and view analytics covering revenue trends and visitor-category breakdowns.

3. Project Objectives

The project set out to achieve eleven objectives, all of which have been realised and verified. These objectives span the full software engineering lifecycle, from requirements analysis through deployment.

#	Objective	Status
1	Analyse the requirements of visitors and administrators of a tourism attraction booking system	Achieved
2	Design a database for managing users, attractions, bookings, and visit plans	Achieved
3	Develop a secure user authentication mechanism (Manus OAuth)	Achieved
4	Allow visitors to browse, search, and filter available KNMP attractions	Achieved
5	Enable visitors to make and manage visit bookings	Achieved
6	Automatically calculate booking costs based on visitor categories and quantities	Achieved
7	Generate a unique booking reference for each successful booking	Achieved
8	Enable visitors to create and manage a personal visit itinerary	Achieved
9	Provide administrators with a dashboard for managing attractions and bookings	Achieved
10	Test and deploy the completed application	Achieved
11	Identify and document technical debt introduced during development	Achieved (see Technical Debt Plan)

4. Stakeholders and Users

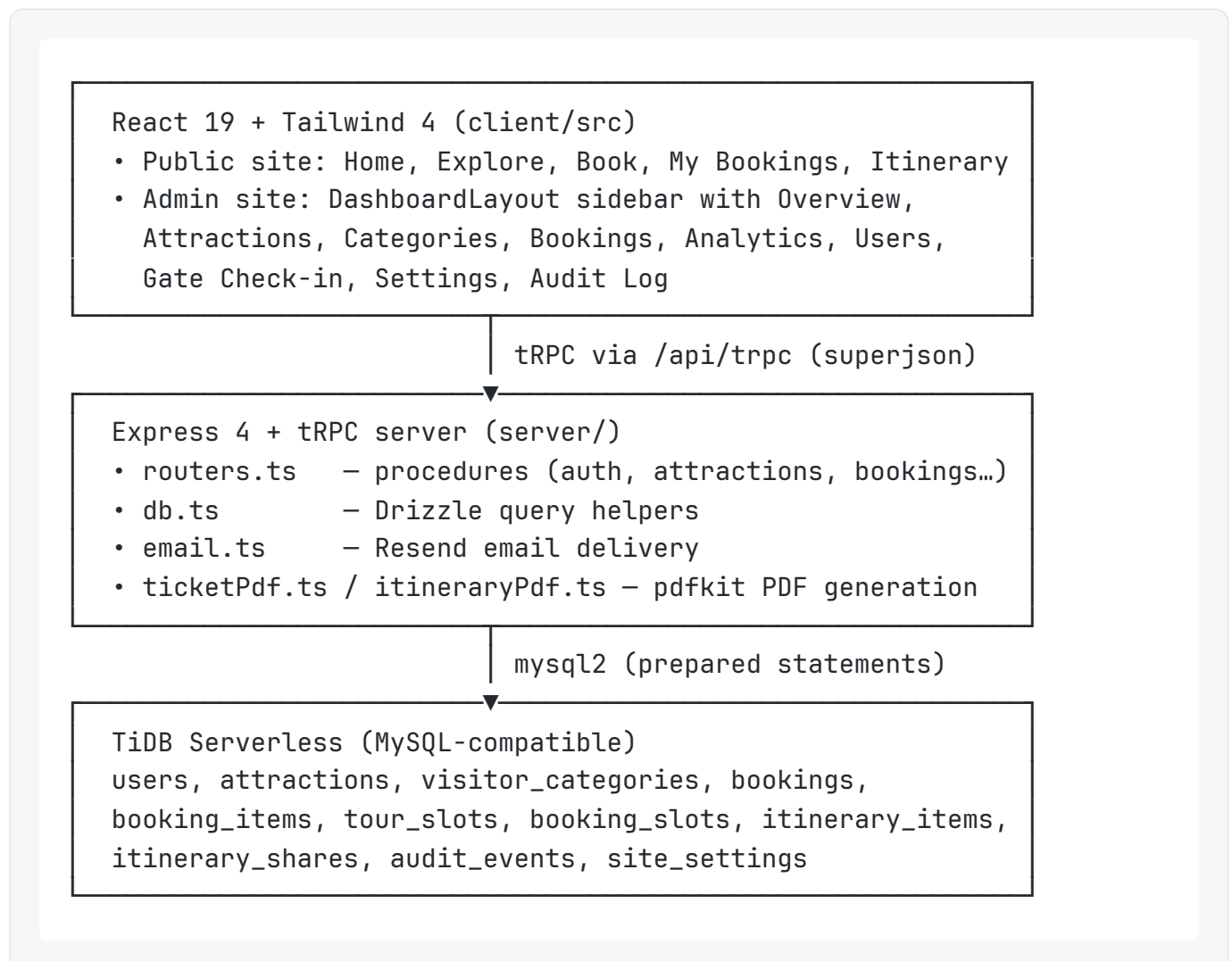
The system serves two primary actors. The **Visitor** is the public-facing user who registers and signs in via Manus OAuth, explores attractions, makes and reviews bookings, downloads tickets,

and builds itineraries. The **Administrator** is a park staff member who manages content, bookings, users, settings, and analytics, and supervises gate check-ins. A system owner (the project proprietor) holds ultimate administrative authority and receives operational oversight.

5. System Architecture

KNMP VisitHub is a modern full-stack TypeScript application. The frontend is built with **React 19**, styled with **Tailwind CSS 4** and **shadcn/ui** components, and routed with **wouter**. The backend is an **Express 4** server exposing a strongly-typed API through **tRPC v11** with **zod** input validation and **superjson** serialization. Data access uses **Drizzle ORM v0.44** over **MySQL (TiDB Serverless)** via **mysql2**. Email delivery uses the **Resend** transactional email API. PDF generation is performed server-side with **pdfkit**, and QR codes are embedded with **qrcode** (server) and decoded client-side with **jsqr** on the gate check-in page.

All traffic is funnelled through a single RPC gateway (`/api/trpc`), which makes the system easy to secure and to route at the edge. Authentication is handled by Manus OAuth with session cookies managed by the framework layer.



6. Database Design

The database comprises twelve tables designed around four domains: identity, content, transactions, and operations.

Table	Domain	Purpose
users	Identity	Accounts, OAuth linkage (<code>openId</code>), role (<code>user</code> / <code>admin</code>), active flag
attractions	Content	Park attractions with description, location, category, images, opening hours
visitor_categories	Content	Fee categories (adult, child, student, foreigner, ...) with prices in pesewas
bookings	Transaction	One row per booking with unique reference, date range, status, check-in timestamp
booking_items	Transaction	Per-category line items (category, quantity, subtotal)
tour_slots	Content	Guided-tour time slots per attraction
booking_slots	Transaction	Slot reservations attached to bookings
itinerary_items	Transaction	Personal visit-plan items (owner-only)
itinerary_shares	Transaction	Public share codes for itineraries
audit_events	Operations	Immutable log of administrative actions
site_settings	Operations	Configurable park-wide settings (capacity, threshold)

Key relationships: a booking belongs to one user and holds many `booking_items` and `booking_slots` ; an itinerary belongs to one user and holds many `itinerary_items` ; attraction categories and locations are stored directly on `attractions` to power the explorer facets. Monetary values are stored as integer pesewas to avoid floating-point currency errors.

7. Implemented Features

The feature set grew through seven enhancement rounds on top of the initial build.

Round 0 — Core platform. Attractions explorer with search; category-based fee structure with live cost calculation; booking flow issuing unique `KNMP-XXXXXX` references; Manus OAuth sign-in; My Bookings with cancellation; personal itinerary builder; admin dashboard for attractions, categories, bookings, and statistics.

Round 1. Multi-day bookings, guided-tour time slot selection, PDF entry tickets, and the first admin analytics page with category breakdowns and monthly revenue trends.

Round 2. User management: registered-users table, role promotion/demotion (with self-protection), and account activation/deactivation with blocked-user authentication gating.

Round 3. Audit logging for role changes and account status changes, plus category and location filter facets on the explorer.

Round 4. Itinerary PDF export and shareable links, extended audit coverage for content and booking-status changes, and date-range filtering across all analytics.

Round 5. Automatic PDF-ticket confirmation emails via Resend, CSV export of the category breakdown, and a daily visitor-capacity forecast chart.

Round 6. Booking status-change emails (confirmed/cancelled), scannable QR codes embedded on PDF tickets, and capacity warnings on the public booking form.

Round 7. Gate check-in page with QR scanning (camera via jsqr and manual reference entry), check-in tracking with badges and a filter on the bookings table, an admin Settings page configuring daily capacity and the near-capacity threshold, and an Add User button with role selection.

8. Security Model

Authentication is delegated to Manus OAuth; the server never stores passwords. Every protected endpoint validates the session cookie, and inactive users are rejected before procedure logic runs. Administrative endpoints are gated by an `adminProcedure` middleware that enforces the `admin` role. Data access follows owner-scoping for visitor actions (a visitor can only read their own bookings and itinerary), and destructive administrative actions write audit events recording the actor, action, target, and detail. Email-sending failures are logged but never block booking or status operations.

9. Development Methodology

The project followed an iterative, checkpoint-driven workflow. Each enhancement round comprised schema design, migration, backend procedures, frontend integration, visual verification, and a Vitest test pass before a checkpoint was saved. Seven checkpoints were released in total, with the final state holding 77 passing tests across six test files. A notable

quality event occurred during Round 4 when a TiDB prepared-statement incompatibility (SELECT-list aliases in `ORDER BY`) surfaced in production queries; it was diagnosed through incremental reproduction, fixed by positional ordering, and covered by regression tests.

10. Deployment

The application is deployed on Manus hosting with autoscaling serverless instances, serving the React build from the same Express service behind the `/api/trpc` gateway. The database is a TiDB Serverless cluster in the US East region. Email delivery is provisioned through a Resend API key stored as a deployment secret. The live deployment URL and source-export instructions are provided in `Deployment_and_Source_Links.txt` .